

VeilleTech

Application de veille scientifique IA

Documentation fonctionnelle

Projet	Veille automatique d'articles scientifiques IA
Langage	Python 3.13
Environnement	Conda — StageS7 (environment.yml)
LLM local	Llama 3 via Ollama (renommage clusters + résumés)
Modèle d'embeddings	all-MiniLM-L6-v2 (sentence-transformers)
Sources	arXiv + Crossref (API publiques, sans clé)
Interface	Flask — web local + tunnel ngrok
Sortie	papers.json + veille_ai.xml (flux RSS)
Lancement Windows	lancer_veille.bat — env. Conda StageS7
Fichiers principaux	collector.py, scorer.py, clustering.py, resum.py, app.py, Run_pipeline.py, EsLatent.py

1. Présentation générale

1.1 Objectif

VeilleTech est une application Python autonome qui automatise la collecte, le scoring, le résumé et la visualisation d'articles scientifiques dans le domaine de l'intelligence artificielle. Elle interroge les API publiques arXiv et Crossref, classe les articles par pertinence grâce à des embeddings sémantiques, génère des résumés en français via un LLM local (Llama 3 / Ollama), et expose les résultats via une interface web Flask accessible en local ou via un tunnel ngrok.

Cas d'usage principal

Un chercheur ou étudiant configure ses thématiques (clusters) dans l'interface, lance le pipeline en un clic, et reçoit dans son navigateur une sélection des 10 articles les plus pertinents par cluster, avec résumés en français, scores de pertinence, et flux RSS exportable.

1.2 Architecture en 5 modules

- collector.py — Collecte des articles depuis arXiv et Crossref
- scorer.py — Scoring sémantique par embeddings (sentence-transformers)
- clustering.py — Renommage automatique des clusters par Llama
- resum.py — Génération de résumés en français par Llama (1 appel par cluster)
- app.py — Serveur Flask : API REST + interface web + orchestration du pipeline
- Run_pipeline.py — Point d'entrée CLI : exécute le pipeline complet puis lance le serveur
- EsLatent.py — Visualisation de l'espace latent (PCA, t-SNE, UMAP optionnel)
- rss.py — Export du flux RSS (format XML)

2. Module collector.py — Collecte des articles

2.1 Principe

Pour chaque cluster défini, le collecteur interroge simultanément deux sources académiques publiques : l'API Atom d'arXiv (preprints) et l'API REST Crossref (articles publiés). Les résultats sont fusionnés et dédoublés par URL avant d'être retournés au pipeline.

2.2 Clusters par défaut

4 clusters thématiques sont prédéfinis dans le code. Ils sont chargés depuis clusters.json s'il existe, sinon les valeurs par défaut sont utilisées et le fichier est créé automatiquement.

ID	Label	Mots-clés principaux
0	Large Language Models	LLM, GPT, transformer, instruction tuning, RLHF, fine-tuning, prompt, tokenizer
1	Generative Models & Diffusion	diffusion model, GAN, VAE, image synthesis, text-to-image, latent diffusion
2	Reinforcement Learning	RL, reward, policy gradient, Q-learning, actor critic, multi-agent, offline RL
3	Graph & Geometric Learning	GNN, graph transformer, geometric deep learning, knowledge graph, message passing

2.3 Sources interrogées

Propriété	Détail
arXiv	API Atom (export.arxiv.org) — tri par date de soumission décroissante — 25 articles max par cluster
Crossref	API REST — filtre type:journal-article — tri par date de publication décroissante — 25 articles max
Déduplication	Par URL — les doublons inter-sources sont supprimés avant scoring
Données retournées	title, abstract, url, source (arXiv ou Crossref)

Aucune clé API requise

Les deux sources (arXiv et Crossref) sont publiques et gratuites. Aucune authentification n'est nécessaire. La seule contrainte est le délai de réponse réseau (timeout 10s par requête).

3. Module scorer.py — Scoring sémantique

3.1 Modèle utilisé

Le scoring utilise le modèle all-MiniLM-L6-v2 de la bibliothèque sentence-transformers. Ce modèle léger (22M paramètres) projette des textes dans un espace vectoriel de dimension 384, optimisé pour la similarité sémantique.

3.2 Algorithme de scoring

L'approche choisit le maximum de similarité entre l'article et chacun des mots-clés du cluster, plutôt qu'une moyenne. Ce choix est délibéré : un article pertinent n'a pas besoin de couvrir tous les mots-clés — il suffit qu'il en frappe un avec une forte similarité.

Propriété	Détail
Étape 1	Encodage de chaque mot-clé individuellement → vecteurs kw_vecs ($n_kw \times 384$)
Étape 2	Encodage de chaque article (titre + abstract concaténés) → vecteurs paper_vecs ($n_papers \times 384$)
Étape 3	Calcul de la matrice de similarité cosinus ($n_papers \times n_kw$)
Score final	Maximum des similarités de l'article avec tous les mots-clés (<code>sim_matrix[i].max()</code>)
Résultat	Top 10 articles par cluster (<code>top_k=10</code>), triés par score décroissant

Choix du maximum vs moyenne

La stratégie max est plus robuste que la moyenne pour les articles spécialisés : un paper sur le fine-tuning de LLMs aura une très haute similarité avec « fine-tuning » mais une faible avec « tokenizer ». La moyenne pénaliserait ce paper — le max le valorise correctement.

4. Module clustering.py — Renommage par LLM

4.1 Rôle

Après scoring, les clusters portent encore leurs labels génériques définis dans clusters.json. Ce module appelle Llama 3 (via Ollama en local) pour renommer chaque cluster avec un label court et précis, basé sur les titres réels des articles sélectionnés.

4.2 Prompt utilisé

Le LLM reçoit les 8 premiers titres d'articles du cluster (pour limiter le contexte) et doit répondre avec uniquement un label de 2 à 4 mots, sans ponctuation ni explication. Ce label remplace ensuite le champ label dans le cluster et dans tous ses articles.

Propriété	Détail
Modèle	llama3 (Ollama local — aucune API externe)
Entrée	Jusqu'à 8 titres d'articles du cluster
Sortie attendue	Label court (2-4 mots) — ex: « LLM Instruction Tuning »
Robustesse	Seule la première ligne de la réponse est conservée (splitlines()[0])
Impact	Le nouveau label est propagé à tous les articles du cluster (champ cluster_title)

4.3 Dépendance Ollama

Prérequis : Ollama installé et llama3 téléchargé

Ce module nécessite qu'Ollama soit lancé localement (ollama serve) et que le modèle llama3 soit disponible (ollama pull llama3). Si Ollama n'est pas disponible, le pipeline lèvera une exception à l'étape [3/5].

5. Module resum.py — Résumés en français

5.1 Stratégie batch

Pour minimiser le nombre d'appels LLM, tous les articles d'un cluster sont résumés en un seul appel Llama. Le prompt envoie tous les abstracts numérotés [0], [1], ... et demande un tableau JSON de résumés dans le même ordre.

Propriété	Détail
Modèle	llama3 (Ollama local)
Approche	1 appel par cluster (batch) — pas 1 appel par article
Langue de sortie	Français (spécifié dans le prompt)
Format attendu	JSON array — exactement N éléments (N = nombre d'articles)
Nettoyage	Suppression des balises ``json ... `` avant parsing
Fallback	Si le JSON est malformé : extraction par regex des chaînes entre guillemets, puis découpage ligne par ligne
Valeur par défaut	« Résumé non disponible. » si l'index est absent ou vide

5.2 Robustesse du parsing

La méthode `_fallback_parse` tente deux approches successives si `JSON.loads` échoue : extraction par regex de toutes les chaînes entre guillemets dans la réponse brute, puis découpage par lignes. Le pipeline ne s'interrompt jamais pour un échec de résumé — le champ `summary` est simplement marqué « Résumé non disponible. ».

6. Module app.py — Serveur Flask et API REST

6.1 Routes disponibles

Méthode	Endpoint	Description
GET	/	Page principale — charge papers.json, groupe par cluster, injecte dans le template Jinja2
GET	/api/clusters	Retourne la liste des clusters (clusters.json)
POST	/api/clusters	Sauvegarde une nouvelle configuration de clusters (body JSON)
GET	/api/papers	Retourne les articles groupés par cluster_id (depuis papers.json)
POST	/api/pipeline/start	Lance le pipeline complet dans un thread daemon en arrière-plan
GET	/api/pipeline/status	Retourne l'état du pipeline : running, done, error, log (liste de messages)

6.2 Pipeline asynchrone

Le pipeline est exécuté dans un thread daemon Python (`threading.Thread`) pour ne pas bloquer le serveur web pendant le traitement. Le statut est stocké dans le dictionnaire global `_pipeline_status` et consultable en temps réel via `/api/pipeline/status`.

Propriété	Détail
État initial	{running: False, done: False, error: False, log: []}
Pendant l'exécution	{running: True, log: [messages en temps réel]}
En fin de pipeline	{running: False, done: True, log: [résumé final]}
En cas d'erreur	{error: True, log: [message d'erreur]}
Sécurité	Le démarrage est refusé (409 Conflict) si un pipeline est déjà en cours

6.3 Gestion de papers.json

À chaque exécution du pipeline, les nouveaux articles sont fusionnés avec les anciens (les deux chargés depuis papers.json). La déduplication est faite par URL. Le fichier est ensuite trié par score décroissant et tronqué à 400 articles maximum pour éviter une croissance illimitée.

Persistance des données

papers.json est le fichier central de persistance. Il accumule les résultats de toutes les exécutions passées. Pour repartir d'une base vide, il suffit de supprimer ce fichier avant un nouveau lancement du pipeline.

7. Module Run_pipeline.py — Point d'entrée principal

7.1 Séquence d'exécution

Run_pipeline.py orchestre l'ensemble du processus en 5 étapes séquentielles, affiche les logs dans le terminal, puis démarre le serveur Flask avec un tunnel ngrok pour un accès externe.

Étape	Nom	Description
0	Chargement clusters	Charge clusters.json ou utilise les 4 clusters par défaut
1/5	Collecte	fetch_papers_by_cluster() — 25 articles max par source par cluster
2/5	Scoring	score_papers_for_cluster() — top 10 par cluster via embeddings
3/5	Renommage	rename_clusters() — 1 appel Llama par cluster
4/5	Résumés	summarize_cluster() — 1 appel Llama par cluster (batch)
5/5	Sauvegarde	Merge avec papers.json existant + génération veille_ai.xml
Post-pipeline	Serveur web	Tunnel ngrok + Flask sur port 5000 + ouverture navigateur

7.2 Tunnel ngrok

Après le pipeline, un tunnel ngrok est ouvert pour rendre l'interface accessible depuis n'importe quel navigateur via une URL publique fixe. Le domaine est configuré en dur dans le code.

Propriété	Détail
Domaine ngrok	liberty-unarrogant-alaya.ngrok-free.dev (domaine statique gratuit)
Port local	5000 (Flask)
Fallback	Si ngrok échoue, accès local uniquement via http://127.0.0.1:5000
Ouverture auto	Le navigateur s'ouvre automatiquement après 1,5 secondes (thread daemon)

7.3 Lancement Windows — lancer_veille.bat

Le fichier .bat active l'environnement Conda StageS7, se déplace dans le dossier du projet et lance Run_pipeline.py. La commande pause maintient la fenêtre ouverte après exécution pour voir les logs.

```
@echo off
cd /d C:\Users\yayaa\VeilleTech-code
call C:\Users\yayaa\anaconda3\Scripts\activate.bat StageS7
python Run_pipeline.py
pause
```

Adaptation nécessaire

Le chemin C:\Users\yayaa\ est spécifique à la machine de développement. Pour déployer sur un autre poste, remplacer yayaa par le nom d'utilisateur Windows correspondant dans lancer_veille.bat.

8. Module EsLatent.py — Visualisation de l'espace latent

8.1 Objectif

Ce module génère une visualisation interactive de l'espace vectoriel dans lequel sont projetés les mots-clés et les articles. Il permet de vérifier visuellement la cohérence des clusters — les articles d'un même thème doivent se regrouper dans l'espace sémantique.

8.2 Projections disponibles

Propriété	Détail
PCA	Analyse en Composantes Principales — projection linéaire rapide, préserve la variance globale
t-SNE	t-Distributed Stochastic Neighbor Embedding — non linéaire, préserve les proximités locales
UMAP	Uniform Manifold Approximation — optionnel (pip install umap-learn) — plus rapide que t-SNE

8.3 Fonctionnement

- Charge clusters.json et papers.json
- Encode chaque mot-clé (rond ●) et chaque article (carré ■) avec all-MiniLM-L6-v2
- Projette tous les vecteurs en 2D avec chaque méthode
- Affiche une figure matplotlib avec couleur par cluster et forme par type (mot-clé vs article)
- Tooltip au survol de la souris (label du point)
- Sauvegarde la figure en latent_space.png

Usage recommandé

Lancer EsLatent.py après un pipeline complet pour vérifier que les clusters sont bien séparés dans l'espace latent. Si deux clusters sont confondus, leurs mots-clés sont sémantiquement trop proches — il faut les reformuler.

9. Fichiers de données

9.1 clusters.json

Fichier de configuration des thématiques de veille. Généré automatiquement depuis les valeurs par défaut si absent. Modifiable directement via l'interface web (POST /api/clusters) ou manuellement.

```
[ { "id": 0, "label": "Large Language Models",  
  "keywords": ["large language model", "LLM", ...],  
  "query_str": "large language model OR LLM OR ..." } ]
```

9.2 papers.json

Base de données principale des articles collectés. Accumulatif — les nouvelles exécutions ajoutent leurs articles aux anciens (déduplication par URL). Trié par score décroissant, limité à 400 entrées.

Propriété	Détail
Champs par article	title, abstract, url, source, cluster (id), cluster_title, score, summary
Taille maximale	400 articles (les moins bien scorés sont éliminés lors de chaque mise à jour)
Localisation	Racine du projet (même dossier que app.py)

9.3 veille_ai.xml

Flux RSS généré par rss.py à chaque fin de pipeline. Contient les 50 premiers articles (par ordre de score). Chaque entrée inclut le titre, le lien, la source et le score de pertinence. Le flux peut être importé dans tout agrégateur RSS compatible.

10. Environnement et dépendances

10.1 Environnement Conda

Propriété	Détail
Nom	StageS7
Python	3.13.11
Gestionnaire	Anaconda (conda-forge + defaults)
Fichier de config	environment.yml (à la racine du projet)

10.2 Dépendances principales (pip)

Bibliothèque	Version	Rôle
flask	3.1.0	Serveur web et API REST
sentence-transformers	5.2.3	Embeddings sémantiques (all-MiniLM-L6-v2)
torch	2.10.0	Backend tensor pour sentence-transformers
ollama	0.6.1	Client Python pour Llama 3 local
feedgen	1.0.0	Génération du flux RSS XML
scikit-learn	1.8.0	PCA, t-SNE pour visualisation
matplotlib	(via conda)	Visualisation de l'espace latent
numpy	2.4.2	Calculs vectoriels
requests	2.32.5	Appels HTTP vers arXiv et Crossref
pyngrok	(via pip)	Tunnel ngrok pour accès externe
umap-learn	(optionnel)	Projection UMAP — pip install umap-learn

10.3 Installation

1. Installer Anaconda et créer l'environnement : `conda env create -f environment.yml`
2. Installer Ollama : <https://ollama.com> — puis `ollama pull llama3`
3. Configurer un compte ngrok gratuit et un domaine statique (ou désactiver ngrok dans `Run_pipeline.py`)
4. Lancer : double-clic sur `lancer_veille.bat` (Windows) ou `python Run_pipeline.py`

11. Glossaire

Terme	Définition
Cluster	Groupe thématique défini par un label et des mots-clés. Sert à organiser la collecte et le scoring.
Embedding	Représentation vectorielle d'un texte dans un espace à haute dimension. Deux textes proches sémantiquement ont des vecteurs proches.
Similarité cosinus	Mesure de similitude entre deux vecteurs — 1 = identiques, 0 = orthogonaux (aucun lien). Utilisée pour scorer les articles.
all-MiniLM-L6-v2	Modèle de sentence-transformers optimisé pour la similarité sémantique. 22M paramètres, espace de dimension 384.
Ollama	Outil qui permet de faire tourner des LLMs en local (sans API cloud). Utilisé ici pour Llama 3.
Llama 3	Modèle de langage open-source de Meta. Utilisé pour le renommage des clusters et les résumés d'articles.
arXiv	Archive ouverte de preprints scientifiques (physique, maths, informatique, IA...). API gratuite et publique.
Crossref	Index de métadonnées d'articles publiés dans des revues académiques. API gratuite et publique.
Flask	Micro-framework web Python. Utilisé pour servir l'interface HTML et l'API REST.
ngrok	Service de tunnel réseau qui expose un port local vers une URL HTTPS publique.
PCA	Analyse en Composantes Principales — réduction de dimension linéaire qui préserve la variance maximale.
t-SNE	Algorithme de réduction non-linéaire qui préserve la structure locale des données (voisins proches).
UMAP	Alternative à t-SNE, plus rapide, qui préserve aussi la structure globale des données.
RSS	Format XML de syndication de contenu web. Permet d'abonner des outils de veille au flux d'articles générés.
papers.json	Base de données locale accumulant tous les articles collectés et analysés par le pipeline.